

garantiu — Site Completo (7 telas) Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (- []) syntax for tracking.

Goal: Build a working Python + Streamlit version de todas as 7 telas do wireframe (docs/wireframe/garantiu-wireframe.html) — Conectar Release, Visão Geral do Risco, Roteiro de Teste Manual, Suíte Automatizada Priorizada, Detalhe do Módulo, Decisão de Publicação e Histórico & Tendências — com dado real (Git, relatório JUnit, histórico de execuções de teste, CSV de incidentes), não mock estático.

Architecture: Uma biblioteca de funções puras e testáveis (`garantiu/`) faz toda leitura de dado e cálculo; um único script Streamlit (`app.py`) liga essas funções às 7 telas via `st.session_state` . Duas fontes de estado persistente em SQLite: o log de decisões de publicação (Task 7) e, agora, o histórico de execuções de teste pra calcular flakiness real (Task 9) e o histórico de score x resultado real por release (Task 14). Sem framework web além do Streamlit — sem split front/back separado.

Tech Stack: Python 3.10+, Streamlit (UI), GitPython (Git history), junitparser (JUnit XML test reports), sqlite3 (stdlib — decisões, histórico de testes, histórico de releases), pytest (tests).

Global Constraints

- Este plano cobre as **7 telas completas** do wireframe: 1 Conectar Release, 2 Visão Geral do Risco, 3 Roteiro de Teste Manual, 4 Suíte Automatizada Priorizada, 5 Detalhe do Módulo, 6 Decisão de Publicação, 7 Histórico & Tendências.
- Score por módulo = média ponderada de 4 fatores, pesos iguais (25% cada): `complexidade` , `bugs` , `saude_testes` , `incidentes` . Score do release = `max()` dos scores dos módulos alterados (ver docs/superpowers/specs/2026-09-09-garantiu-wireframe-design.md).
- `saude_testes` (a partir da Task 9) combina 50% taxa de falha da rodada atual + 50% flakiness histórica real, calculada a partir de múltiplas execuções armazenadas — não é mais um placeholder de rodada única.
- Módulo sem teste correspondente no relatório recebe `saude_testes = 0` (sem risco adicional por falta de dado), não `100` de risco.
- **Decisão de arquitetura, não simplificação temporária:** o fator `incidentes` vem de um arquivo local (CSV) preenchido pela equipe, não de uma integração ao vivo com Jira/rastreador — é a mesma decisão de "formatos genéricos" já tomada pra Git (diff puro) e testes (JUnit XML) em vez de amarrar o produto a uma ferramenta específica.
- `complexidade` , `bugs` e `incidentes` são normalizados (0–100) relativos ao maior valor **entre os módulos alterados neste release** — não contra o histórico completo do repositório.
- Flakiness (Task 9) e histórico de releases (Task 14) exigem múltiplas execuções/análises registradas ao longo do tempo — na primeira vez que o app roda contra um repositório novo, esses dados começam vazios (0% flakiness, sem histórico) e vão se populando a cada análise. Isso é esperado, não é bug.
- Todo código Python segue PEP 8 padrão; sem dependências além das listadas no Tech Stack.
- Todas as funções em `garantiu/` são puras (sem `print` , sem estado global) para serem testáveis sem subir o Streamlit.

File Structure

```
garantiu/
  __init__.py
  git_reader.py          # Task 1 – lê arquivos alterados entre duas referências Git
  bug_history.py         # Task 2 – minera commits de correção de bug
  test_reports.py        # Task 3 – lê relatório JUnit XML
  incidents.py           # Task 4 – lê CSV de incidentes
  scoring.py             # Task 5 – combina os 4 fatores no score
  manual_test_guide.py   # Task 6 – gera os cards da tela "Roteiro de Teste Manual"
  decision_log.py        # Task 7 – grava/lê decisões de publicação (SQLite)
  test_history.py        # Task 9 – grava execuções de teste e calcula flakiness real
  test_prioritization.py # Task 10 – ordena a suíte automatizada por risco (tela 4)
  module_detail.py       # Task 12 – agrega diff + bugs + incidentes + testes de 1 módulo (tela 5)
  release_history.py     # Task 13 – grava score previsto x resultado real por release (tela 7)
app.py                  # Task 8 (telas 1,2,3,6) + Task 14 (telas 4,5,7) – Streamlit
sample_data/
  incidents.csv          # Task 8 – contagem de incidentes por módulo
  incident_details.csv   # Task 11 – descrição/data de cada incidente (tela 5)
  sample_junit.xml       # Task 8 – dado de exemplo pra rodar a demo
tests/
  __init__.py
  test_git_reader.py
  test_bug_history.py
  test_test_reports.py
  test_incidents.py
  test_scoring.py
  test_manual_test_guide.py
  test_decision_log.py
  test_app_smoke.py
  test_test_history.py
  test_test_prioritization.py
  test_module_detail.py
  test_release_history.py
requirements.txt         # Task 1
```

Task 1: Scaffold do projeto + leitura do Git

Files: - Create: requirements.txt - Create: garantiu/__init__.py - Create: garantiu/git_reader.py - Create: tests/__init__.py - Test: tests/test_git_reader.py

Interfaces: - Produces: get_changed_files(repo_path: str, base_ref: str, head_ref: str) -> list[dict] , onde cada dict tem as chaves "path" , "module" , "lines_added" , "lines_removed" .

- ❑ **Step 1: Criar** requirements.txt

```
streamlit>=1.32
GitPython>=3.1
junitparser>=3.1
pytest>=8.0
```

- ❑ **Step 2: Instalar dependências**

Run: pip install -r requirements.txt Expected: instalação sem erro.

- ❑ **Step 3: Criar os pacotes vazios**

garantiu/__init__.py — arquivo vazio. tests/__init__.py — arquivo vazio.

- ❑ **Step 4: Escrever o teste que falha**

tests/test_git_reader.py :

```
import git
import pytest

from garantiu.git_reader import get_changed_files

@pytest.fixture
def sample_repo(tmp_path):
    repo_path = tmp_path / "repo"
    repo_path.mkdir()
    repo = git.Repo.init(repo_path)
    (repo_path / "checkout").mkdir()
    gateway = repo_path / "checkout" / "gateway.py"
    gateway.write_text("def pay():\n    pass\n")
    repo.index.add(["checkout/gateway.py"])
    repo.index.commit("initial commit")
    repo.create_tag("v1.0.0")

    gateway.write_text("def pay():\n    pass\n\ndef refund():\n    pass\n")
    repo.index.add(["checkout/gateway.py"])
    repo.index.commit("add refund")
    return str(repo_path)

def test_get_changed_files_detects_module_and_lines(sample_repo):
    changes = get_changed_files(sample_repo, "v1.0.0", "HEAD")
    assert len(changes) == 1
    assert changes[0]["path"] == "checkout/gateway.py"
    assert changes[0]["module"] == "checkout"
    assert changes[0]["lines_added"] == 2
    assert changes[0]["lines_removed"] == 0

def test_get_changed_files_uses_filename_as_module_at_repo_root(tmp_path):
    repo_path = tmp_path / "repo2"
    repo_path.mkdir()
    repo = git.Repo.init(repo_path)
    f = repo_path / "README.md"
    f.write_text("v1")
    repo.index.add(["README.md"])
    repo.index.commit("initial")
    repo.create_tag("v1.0.0")
    f.write_text("v1\nv2")
    repo.index.add(["README.md"])
    repo.index.commit("update readme")

    changes = get_changed_files(str(repo_path), "v1.0.0", "HEAD")
    assert changes[0]["module"] == "README.md"
```

- ❑ **Step 5: Rodar o teste e confirmar que falha**

Run: `pytest tests/test_git_reader.py -v` Expected: FAIL com `ModuleNotFoundError: No module named 'garantiu.git_reader'`

- ❑ **Step 6: Implementar** `garantiu/git_reader.py`

```
import git

def get_changed_files(repo_path: str, base_ref: str, head_ref: str) -> list[dict]:
    """
    Returns changed files between base_ref and head_ref as a list of dicts:
    {"path": str, "module": str, "lines_added": int, "lines_removed": int}.
    module is the top-level directory of path, or the filename itself if the
    file lives at the repo root.
    """
    repo = git.Repo(repo_path)
    numstat = repo.git.diff(base_ref, head_ref, "--numstat")

    results = []
    for line in numstat.splitlines():
        if not line.strip():
            continue
        added_str, removed_str, path = line.split("\t")
        added = 0 if added_str == "-" else int(added_str)
        removed = 0 if removed_str == "-" else int(removed_str)
        module = path.split("/")[0] if "/" in path else path
        results.append({
            "path": path,
            "module": module,
            "lines_added": added,
            "lines_removed": removed,
        })
    return results
```

- ☐ **Step 7: Rodar o teste e confirmar que passa**

Run: `pytest tests/test_git_reader.py -v` Expected: 2 passed

- ☐ **Step 8: Commit**

```
git add requirements.txt garantiu/__init__.py garantiu/git_reader.py tests/__init__.py tests/test_git_reader.py
git commit -m "feat: read changed files between two git refs"
```

Task 2: Histórico de bugs (mineração de commits)

Files: - Create: `garantiu/bug_history.py` - Test: `tests/test_bug_history.py`

Interfaces: - Consumes: nada de tasks anteriores (lê o repositório Git diretamente). - Produces:

`get_bug_fix_commits(repo_path: str) -> list[dict]` (chaves "hash", "message", "files"); `build_bug_history(repo_path: str) -> dict[str, int]` (mapa path -> contagem).

- ☐ **Step 1: Escrever o teste que falha**

`tests/test_bug_history.py` :

```

import git
import pytest

from garantiu.bug_history import build_bug_history, get_bug_fix_commits

@pytest.fixture
def repo_with_bug_fixes(tmp_path):
    repo_path = tmp_path / "repo"
    repo_path.mkdir()
    repo = git.Repo.init(repo_path)
    (repo_path / "checkout").mkdir()
    f = repo_path / "checkout" / "gateway.py"

    f.write_text("v1")
    repo.index.add(["checkout/gateway.py"])
    repo.index.commit("initial commit")

    f.write_text("v2")
    repo.index.add(["checkout/gateway.py"])
    repo.index.commit("fix: corrige calculo do gateway")

    f.write_text("v3")
    repo.index.add(["checkout/gateway.py"])
    repo.index.commit("add new feature")

    f.write_text("v4")
    repo.index.add(["checkout/gateway.py"])
    repo.index.commit("fixes bug in refund flow")

    return str(repo_path)

def test_get_bug_fix_commits_filters_by_message(repo_with_bug_fixes):
    commits = get_bug_fix_commits(repo_with_bug_fixes)
    assert len(commits) == 2
    messages = [c["message"] for c in commits]
    assert any("corrige" in m for m in messages)
    assert any("fixes bug" in m for m in messages)

def test_build_bug_history_counts_per_file(repo_with_bug_fixes):
    history = build_bug_history(repo_with_bug_fixes)
    assert history["checkout/gateway.py"] == 2

```

- ☐ **Step 2: Rodar o teste e confirmar que falha**

Run: `pytest tests/test_bug_history.py -v` Expected: FAIL com `ModuleNotFoundError`

- ☐ **Step 3: Implementar** `garantiu/bug_history.py`

```

import re

import git

BUG_COMMIT_PATTERN = re.compile(
    r"\b(fix|fixes|fixed|closes|resolve|resolves|bug|corrige|corrigido)\b",
    re.IGNORECASE,
)

def get_bug_fix_commits(repo_path: str) -> list[dict]:
    """
    Scans the full commit history and returns commits whose message matches
    BUG_COMMIT_PATTERN, as {"hash": str, "message": str, "files": list[str]}.
    """
    repo = git.Repo(repo_path)
    results = []
    for commit in repo.iter_commits():
        if BUG_COMMIT_PATTERN.search(commit.message):
            results.append({
                "hash": commit.hexsha,
                "message": commit.message.strip(),
                "files": list(commit.stats.files.keys()),
            })
    return results

def build_bug_history(repo_path: str) -> dict:
    """Returns {file_path: count_of_bug_fix_commits_touching_it}."""
    history: dict = {}
    for commit in get_bug_fix_commits(repo_path):
        for path in commit["files"]:
            history[path] = history.get(path, 0) + 1
    return history

```

- ☐ **Step 4: Rodar o teste e confirmar que passa**

Run: `pytest tests/test_bug_history.py -v` Expected: 2 passed

- ☐ **Step 5: Commit**

```

git add garantiu/bug_history.py tests/test_bug_history.py
git commit -m "feat: mine bug-fix commits and per-file bug history"

```

Task 3: Leitura do relatório de testes (JUnit XML)

Files: - Create: `garantiu/test_reports.py` - Test: `tests/test_test_reports.py`

Interfaces: - Consumes: nada de tasks anteriores. - Produces: `parse_junit_report(xml_path: str) -> list[dict]` (chaves "name", "classname", "status", "time"); `test_health_by_module(test_results: list[dict]) -> dict[str, float]`.

- ☐ **Step 1: Escrever o teste que falha**

`tests/test_test_reports.py` :

```

import pytest

from garantiu.test_reports import parse_junit_report, test_health_by_module

SAMPLE_XML = """<?xml version="1.0" encoding="utf-8"?>
<testsuites>
  <testsuite name="pagamentos" tests="3">
    <testcase classname="checkout.test_gateway" name="test_pagamento_aprovado" time="0.01"/>
    <testcase classname="checkout.test_gateway" name="test_pagamento_recusado" time="0.02">
      <failure message="AssertionError">expected True, got False</failure>
    </testcase>
    <testcase classname="auth.test_sessao" name="test_expira_token" time="0.03"/>
  </testsuite>
</testsuites>
"""

@pytest.fixture
def sample_junit_file(tmp_path):
    path = tmp_path / "report.xml"
    path.write_text(SAMPLE_XML)
    return str(path)

def test_parse_junit_report_reads_status(sample_junit_file):
    results = parse_junit_report(sample_junit_file)
    assert len(results) == 3
    statuses = {r["name"]: r["status"] for r in results}
    assert statuses["test_pagamento_aprovado"] == "passed"
    assert statuses["test_pagamento_recusado"] == "failed"
    assert statuses["test_expira_token"] == "passed"

def test_health_by_module_computes_pass_rate(sample_junit_file):
    results = parse_junit_report(sample_junit_file)
    health = test_health_by_module(results)
    assert health["checkout"] == 50.0
    assert health["auth"] == 100.0

```

- ☐ **Step 2: Rodar o teste e confirmar que falha**

Run: `pytest tests/test_test_reports.py -v` Expected: FAIL com `ModuleNotFoundError`

- ☐ **Step 3: Implementar** `garantiu/test_reports.py`

```

from junitparser import JUnitXml, Skipped

def parse_junit_report(xml_path: str) -> list[dict]:
    """
    Parses a JUnit XML report into a list of
    {"name": str, "classname": str, "status": "passed"|"failed"|"skipped", "time": float}.
    """
    xml = JUnitXml.fromfile(xml_path)
    results = []
    for suite in xml:
        for case in suite:
            result = case.result
            if result and isinstance(result[0], Skipped):
                status = "skipped"
            elif result:
                status = "failed"
            else:
                status = "passed"
            results.append({
                "name": case.name,
                "classname": case.classname,
                "status": status,
                "time": case.time or 0.0,
            })
    return results

def test_health_by_module(test_results: list) -> dict:
    """
    Groups results by module (first segment of classname, split on '.') and
    returns {module: health_score} where health_score = 100 * passed / total,
    rounded to 1 decimal place.
    """
    by_module: dict = {}
    for r in test_results:
        module = r["classname"].split(".")[0]
        by_module.setdefault(module, []).append(r)

    health = {}
    for module, cases in by_module.items():
        passed = sum(1 for c in cases if c["status"] == "passed")
        health[module] = round(100 * passed / len(cases), 1)
    return health

```

- ☐ **Step 4: Rodar o teste e confirmar que passa**

Run: `pytest tests/test_test_reports.py -v` Expected: 2 passed

- ☐ **Step 5: Commit**

```

git add garantiu/test_reports.py tests/test_test_reports.py
git commit -m "feat: parse JUnit XML reports and compute test health per module"

```

Task 4: Leitura de incidentes (CSV)

Files: - Create: `garantiu/incidents.py` - Test: `tests/test_incidents.py`

Interfaces: - Produces: `load_incidents(csv_path: str) -> dict[str, int]` .

- ☐ **Step 1: Escrever o teste que falha**

tests/test_incidents.py :

```
import pytest

from garantiu.incidents import load_incidents

@pytest.fixture
def sample_incidents_csv(tmp_path):
    path = tmp_path / "incidents.csv"
    path.write_text("module,incident_count\ncheckout,3\nauth,1\ncatalogo,0\n")
    return str(path)

def test_load_incidents_reads_counts(sample_incidents_csv):
    incidents = load_incidents(sample_incidents_csv)
    assert incidents == {"checkout": 3, "auth": 1, "catalogo": 0}

def test_load_incidents_skips_bad_rows(tmp_path):
    path = tmp_path / "bad.csv"
    path.write_text("module,incident_count\ncheckout,tres\nauth,2\n")
    incidents = load_incidents(str(path))
    assert incidents == {"auth": 2}
```

- ☐ **Step 2: Rodar o teste e confirmar que falha**

Run: `pytest tests/test_incidents.py -v` Expected: FAIL com `ModuleNotFoundError`

- ☐ **Step 3: Implementar** `garantiu/incidents.py`

```
import csv

def load_incidents(csv_path: str) -> dict:
    """
    Reads a CSV with columns 'module' and 'incident_count', returns
    {module: incident_count}. Rows with a missing or non-integer
    incident_count are skipped.
    """
    incidents = {}
    with open(csv_path, newline="", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        for row in reader:
            try:
                incidents[row["module"]] = int(row["incident_count"])
            except (KeyError, ValueError):
                continue
    return incidents
```

- ☐ **Step 4: Rodar o teste e confirmar que passa**

Run: `pytest tests/test_incidents.py -v` Expected: 2 passed

- ☐ **Step 5: Commit**

```
git add garantiu/incidents.py tests/test_incidents.py
git commit -m "feat: load incident counts per module from CSV"
```

Task 5: Motor de score (o coração do produto)

Files: - Create: `garantiu/scoring.py` - Test: `tests/test_scoring.py`

Interfaces: - Consumes: a saída de `get_changed_files` (Task 1), `build_bug_history` (Task 2), `test_health_by_module` (Task 3), `load_incidents` (Task 4), `flakiness_by_module` (Task 9 — passar `{}` até a Task 9 existir). - Produces: `score_modules(changed_files, bug_history, test_health, incidents, flakiness) -> list[dict]` (cada item: `"module"`, `"score"`, `"factors"`); `score_release(module_scores: list[dict]) -> dict ( "score" , "top_module" , "factors" )`.

- ☐ **Step 1: Escrever o teste que falha**

`tests/test_scoring.py` :

```

from garantiu.scoring import normalize_across_modules, score_modules, score_release

def test_normalize_across_modules_scales_relative_to_max():
    result = normalize_across_modules({"a": 10, "b": 5, "c": 0})
    assert result == {"a": 100.0, "b": 50.0, "c": 0.0}

def test_normalize_across_modules_handles_all_zero():
    result = normalize_across_modules({"a": 0, "b": 0})
    assert result == {"a": 0.0, "b": 0.0}

def test_normalize_across_modules_handles_empty():
    assert normalize_across_modules({}) == {}

def test_score_modules_combines_four_factors():
    changed_files = [
        {"path": "checkout/gateway.py", "module": "checkout", "lines_added": 100, "lines_removed": 20},
        {"path": "catalogo/busca.py", "module": "catalogo", "lines_added": 5, "lines_removed": 0},
    ]
    bug_history = {"checkout/gateway.py": 4}
    test_health = {"checkout": 60.0, "catalogo": 100.0}
    incidents = {"checkout": 2}
    flakiness = {"checkout": 20.0, "catalogo": 0.0}

    results = score_modules(changed_files, bug_history, test_health, incidents, flakiness)

    checkout = next(r for r in results if r["module"] == "checkout")
    catalogo = next(r for r in results if r["module"] == "catalogo")

    assert checkout["factors"]["complexidade"] == 100.0
    assert checkout["factors"]["bugs"] == 100.0
    # saude_testes = 0.5 * (100 - health) + 0.5 * flakiness = 0.5*40 + 0.5*20 = 30.0
    assert checkout["factors"]["saude_testes"] == 30.0
    assert checkout["factors"]["incidentes"] == 100.0
    assert checkout["score"] == 82.5
    assert catalogo["score"] < checkout["score"]

def test_score_modules_missing_test_data_means_zero_risk():
    changed_files = [
        {"path": "novo/modulo.py", "module": "novo", "lines_added": 1, "lines_removed": 0},
    ]
    results = score_modules(changed_files, bug_history={}, test_health={}, incidents={}, flakiness={})
    assert results[0]["factors"]["saude_testes"] == 0.0

def test_score_release_uses_max_module_score_regardless_of_order():
    module_scores = [
        {"module": "catalogo", "score": 20.0, "factors": {"complexidade": 10.0}},
        {"module": "checkout", "score": 85.0, "factors": {"complexidade": 100.0}},
    ]
    release = score_release(module_scores)
    assert release["score"] == 85.0
    assert release["top_module"] == "checkout"
    assert release["factors"] == {"complexidade": 100.0}

def test_score_release_handles_empty_list():
    release = score_release([])
    assert release == {"score": 0.0, "top_module": None, "factors": {}}

```

- ☐ **Step 2: Rodar o teste e confirmar que falha**

Run: `pytest tests/test_scoring.py -v` Expected: FAIL com `ModuleNotFoundError`

- ☐ **Step 3: Implementar** `garantiu/scoring.py`

```

WEIGHTS = {
    "complexidade": 0.25,
    "bugs": 0.25,
    "saude_testes": 0.25,
    "incidentes": 0.25,
}

def normalize_across_modules(raw_values: dict) -> dict:
    """
    Scales a {module: raw_value} dict to {module: 0-100} relative to the
    maximum raw_value present. Empty input or an all-zero input returns 0
    for every module (or an empty dict, for empty input).
    """
    if not raw_values:
        return {}
    max_value = max(raw_values.values())
    if max_value == 0:
        return {module: 0.0 for module in raw_values}
    return {
        module: round(100 * value / max_value, 1)
        for module, value in raw_values.items()
    }

def _group_by_module(changed_files: list) -> dict:
    grouped: dict = {}
    for f in changed_files:
        grouped.setdefault(f["module"], []).append(f)
    return grouped

def score_modules(changed_files: list, bug_history: dict, test_health: dict, incidents: dict, flakiness: dict) -> list:
    """
    Full scoring pipeline over the modules touched by changed_files. Returns
    a list of {"module": str, "score": float, "factors": {...}} sorted by
    score descending.

    complexidade, bugs and incidentes are normalized relative to the other
    changed modules in this release. saude_testes blends two already-0-100
    risk signals in equal parts: (100 - health%) from the current run and
    the historical flakiness rate. A module with no matching test/flakiness
    data gets 0 risk on that sub-signal, not 100, so missing data doesn't
    unfairly inflate the score.
    """
    grouped = _group_by_module(changed_files)

    complexidade_raw = {}
    bugs_raw = {}
    incidentes_raw = {}
    saude_testes = {}

    for module, files in grouped.items():
        complexidade_raw[module] = sum(f["lines_added"] + f["lines_removed"] for f in files)
        bugs_raw[module] = sum(bug_history.get(f["path"], 0) for f in files)
        incidentes_raw[module] = incidents.get(module, 0)
        health = test_health.get(module, 100.0)
        failure_risk = 100 - health
        flakiness_risk = flakiness.get(module, 0.0)
        saude_testes[module] = round(0.5 * failure_risk + 0.5 * flakiness_risk, 1)

    complexidade = normalize_across_modules(complexidade_raw)
    bugs = normalize_across_modules(bugs_raw)
    incidentes_scores = normalize_across_modules(incidentes_raw)

```

```

results = []
for module in grouped:
    factors = {
        "complexidade": complexidade[module],
        "bugs": bugs[module],
        "saude_testes": saude_testes[module],
        "incidentes": incidentes_scores[module],
    }
    score = round(sum(factors[name] * weight for name, weight in WEIGHTS.items()), 1)
    results.append({"module": module, "score": score, "factors": factors})

return sorted(results, key=lambda r: r["score"], reverse=True)

def score_release(module_scores: list) -> dict:
    """
    Rolls per-module scores up to a release-level score using the
    "weakest link" rule: the release score is the highest module score.
    Returns {"score": float, "top_module": str | None, "factors": dict}.
    """
    if not module_scores:
        return {"score": 0.0, "top_module": None, "factors": {}}
    top = max(module_scores, key=lambda r: r["score"])
    return {"score": top["score"], "top_module": top["module"], "factors": top["factors"]}

```

- ☐ **Step 4: Rodar o teste e confirmar que passa**

Run: `pytest tests/test_scoring.py -v` Expected: 7 passed

- ☐ **Step 5: Commit**

```

git add garantiu/scoring.py tests/test_scoring.py
git commit -m "feat: compute per-module and release risk score from 4 factors"

```

Task 6: Roteiro de teste manual (geração de linguagem simples)

Files: - Create: `garantiu/manual_test_guide.py` - Test: `tests/test_manual_test_guide.py`

Interfaces: - Consumes: a saída de `score_modules` (Task 5, itens da lista) e `get_changed_files` (Task 1). - Produces: `risk_label(score: float) -> str`; `build_module_card(module: str, score: float, factors: dict, changed_files: list) -> dict` (chaves `"module"`, `"score"`, `"risk"`, `"o_que_mudou"`, `"por_que_testar"`, `"cenarios"`).

- ☐ **Step 1: Escrever o teste que falha**

`tests/test_manual_test_guide.py` :

```

from garantiu.manual_test_guide import build_module_card, risk_label

def test_risk_label_thresholds():
    assert risk_label(85) == "alto"
    assert risk_label(70) == "alto"
    assert risk_label(55) == "medio"
    assert risk_label(40) == "medio"
    assert risk_label(10) == "baixo"

def test_build_module_card_picks_dominant_factor():
    changed_files = [
        {"path": "checkout/gateway.py", "module": "checkout", "lines_added": 1, "lines_removed": 0},
    ]
    factors = {"complexidade": 20.0, "bugs": 90.0, "saude_testes": 10.0, "incidentes": 5.0}

    card = build_module_card("checkout", 85.0, factors, changed_files)

    assert card["risk"] == "alto"
    assert "histórico de problema" in card["por_que_testar"]
    assert "checkout/gateway.py" in card["o_que_mudou"]
    assert len(card["cenarios"]) == 3
    assert "checkout" in card["cenarios"][0]

def test_build_module_card_truncates_long_file_list():
    changed_files = [
        {"path": f"checkout/f{i}.py", "module": "checkout", "lines_added": 1, "lines_removed": 0}
        for i in range(5)
    ]
    factors = {"complexidade": 90.0, "bugs": 10.0, "saude_testes": 10.0, "incidentes": 5.0}

    card = build_module_card("checkout", 60.0, factors, changed_files)

    assert card["o_que_mudou"].startswith("5 arquivo(s)")
    assert "..." in card["o_que_mudou"]

```

- ☐ **Step 2: Rodar o teste e confirmar que falha**

Run: `pytest tests/test_manual_test_guide.py -v` Expected: FAIL com `ModuleNotFoundError`

- ☐ **Step 3: Implementar** `garantiu/manual_test_guide.py`

```

RISK_THRESHOLDS = {"alto": 70, "medio": 40}

FACTOR_MESSAGES = {
    "complexidade": (
        "Foi uma mudança grande nessa área – quanto mais código muda, maior "
        "a chance de algo passar despercebido na revisão."
    ),
    "bugs": (
        "Essa área já teve histórico de problema corrigido antes – isso é "
        "um dos sinais mais fortes de que pode dar problema de novo."
    ),
    "saude_testes": (
        "Os testes automatizados dessa área não estão saudáveis (falhando "
        "ou com pouca cobertura) – não dá pra confiar só neles."
    ),
    "incidentes": (
        "Essa área já causou incidente em produção antes – atenção redobrada."
    ),
}

GENERIC_SCENARIOS = [
    "Testar o fluxo principal de {module} do início ao fim",
    "Testar {module} com uma entrada inválida ou inesperada",
    "Testar o comportamento de {module} depois de uma falha (timeout, erro de rede, etc.)",
]

def risk_label(score: float) -> str:
    """Returns 'alto' if score >= 70, 'medio' if score >= 40, else 'baixo'."""
    if score >= RISK_THRESHOLDS["alto"]:
        return "alto"
    if score >= RISK_THRESHOLDS["medio"]:
        return "medio"
    return "baixo"

def build_module_card(module: str, score: float, factors: dict, changed_files: list) -> dict:
    """
    Builds the plain-language card for one module on the "Roteiro de Teste
    Manual" screen. Picks the highest-scoring factor to explain "por que
    testar isso".
    """
    file_names = [f["path"] for f in changed_files if f["module"] == module]
    preview = ", ".join(file_names[:3])
    if len(file_names) > 3:
        preview += ", ..."

    dominant_factor = max(factors, key=factors.get)

    return {
        "module": module,
        "score": score,
        "risk": risk_label(score),
        "o_que_mudou": f"{len(file_names)} arquivo(s) alterado(s) nesta área ({preview}).",
        "por_que_testar": FACTOR_MESSAGES[dominant_factor],
        "cenarios": [s.format(module=module) for s in GENERIC_SCENARIOS],
    }

```

- ☐ **Step 4: Rodar o teste e confirmar que passa**

Run: `pytest tests/test_manual_test_guide.py -v` Expected: 3 passed

- ☐ **Step 5: Commit**


```
git add garantiu/manual_test_guide.py tests/test_manual_test_guide.py
git commit -m "feat: generate plain-language manual test guide cards"
```

Task 7: Log de decisão de publicação (auditoria)

Files: - Create: `garantiu/decision_log.py` - Test: `tests/test_decision_log.py`

Interfaces: - Produces: `init_db(db_path: str) -> None` ; `record_decision(db_path: str, release: str, score: float, decided_by: str, decision: str) -> int` ; `get_decision_history(db_path: str, release: str) -> list[dict]` .

- ❑ **Step 1: Escrever o teste que falha**

`tests/test_decision_log.py` :

```
import pytest

from garantiu.decision_log import get_decision_history, record_decision

@pytest.fixture
def db_path(tmp_path):
    return str(tmp_path / "decisions.db")

def test_record_decision_rejects_invalid_decision(db_path):
    with pytest.raises(ValueError):
        record_decision(db_path, "release/2026.09", 74.0, "m.silva", "talvez")

def test_record_and_fetch_decision_history(db_path):
    record_decision(db_path, "release/2026.09", 74.0, "m.silva", "publicar")
    record_decision(db_path, "release/2026.09", 74.0, "m.silva", "cancelar")

    history = get_decision_history(db_path, "release/2026.09")

    assert len(history) == 2
    assert history[0]["decision"] == "cancelar"
    assert history[1]["decision"] == "publicar"
    assert all(h["release"] == "release/2026.09" for h in history)

def test_get_decision_history_empty_for_unknown_release(db_path):
    assert get_decision_history(db_path, "release/nunca-existiu") == []
```

- ❑ **Step 2: Rodar o teste e confirmar que falha**

Run: `pytest tests/test_decision_log.py -v` Expected: FAIL com `ModuleNotFoundError`

- ❑ **Step 3: Implementar** `garantiu/decision_log.py`

```

import sqlite3
from datetime import datetime, timezone

def init_db(db_path: str) -> None:
    """Creates the decisions table if it doesn't already exist."""
    conn = sqlite3.connect(db_path)
    try:
        conn.execute(
            """
            CREATE TABLE IF NOT EXISTS decisions (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                release TEXT NOT NULL,
                score REAL NOT NULL,
                decided_by TEXT NOT NULL,
                decision TEXT NOT NULL,
                decided_at TEXT NOT NULL
            )
            """
        )
        conn.commit()
    finally:
        conn.close()

def record_decision(db_path: str, release: str, score: float, decided_by: str, decision: str) -> int:
    """
    Inserts a decision record. decision must be 'publicar' or 'cancelar'.
    Returns the new row's id.
    """
    if decision not in ("publicar", "cancelar"):
        raise ValueError("decision must be 'publicar' or 'cancelar'")

    init_db(db_path)
    conn = sqlite3.connect(db_path)
    try:
        cursor = conn.execute(
            "INSERT INTO decisions (release, score, decided_by, decision, decided_at) "
            "VALUES (?, ?, ?, ?, ?)",
            (release, score, decided_by, decision, datetime.now(timezone.utc).isoformat()),
        )
        conn.commit()
        return cursor.lastrowid
    finally:
        conn.close()

def get_decision_history(db_path: str, release: str) -> list:
    """Returns all decision records for a release, most recent first."""
    init_db(db_path)
    conn = sqlite3.connect(db_path)
    conn.row_factory = sqlite3.Row
    try:
        rows = conn.execute(
            "SELECT * FROM decisions WHERE release = ? ORDER BY decided_at DESC",
            (release,),
        ).fetchall()
        return [dict(row) for row in rows]
    finally:
        conn.close()

```

- ❑ **Step 4: Rodar o teste e confirmar que passa**

Run: `pytest tests/test_decision_log.py -v` Expected: 3 passed

- ❑ **Step 5: Commit**

```
git add garantiu/decision_log.py tests/test_decision_log.py
git commit -m "feat: record and query publish decisions in SQLite"
```

Task 8: Interface Streamlit (as 4 telas do MVP) + dados de exemplo

Files: - Create: `app.py` - Create: `sample_data/incidents.csv` - Create: `sample_data/sample_junit.xml` - Test: `tests/test_app_smoke.py`

Interfaces: - Consumes: todas as funções das Tasks 1–7 pelo nome exato definido acima.

- ❑ **Step 1: Criar os dados de exemplo**

`sample_data/incidents.csv` :

```
module,incident_count
checkout,2
auth,1
catalogo,0
```

`sample_data/sample_junit.xml` :

```
<?xml version="1.0" encoding="utf-8"?>
<testsuites>
  <testsuite name="garantia-demo" tests="4">
    <testcase classname="checkout.test_gateway" name="test_pagamento_aprovado" time="0.01"/>
    <testcase classname="checkout.test_gateway" name="test_pagamento_recusado" time="0.02">
      <failure message="AssertionError">expected True, got False</failure>
    </testcase>
    <testcase classname="auth.test_sessao" name="test_expira_token" time="0.03"/>
    <testcase classname="catalogo.test_busca" name="test_filtro_preco" time="0.01"/>
  </testsuite>
</testsuites>
```

- ❑ **Step 2: Escrever o teste de fumaça que falha**

`tests/test_app_smoke.py` :

```
from streamlit.testing.v1 import AppTest

def test_app_loads_without_exceptions():
    at = AppTest.from_file("app.py")
    at.run()
    assert not at.exception
```

- ❑ **Step 3: Rodar o teste e confirmar que falha**

Run: `pytest tests/test_app_smoke.py -v` Expected: FAIL (arquivo `app.py` ainda não existe)

- ❑ **Step 4: Implementar** `app.py`

```

import streamlit as st

from garantiu.bug_history import build_bug_history
from garantiu.decision_log import get_decision_history, record_decision
from garantiu.git_reader import get_changed_files
from garantiu.incidents import load_incidents
from garantiu.manual_test_guide import build_module_card
from garantiu.scoring import score_modules, score_release
from garantiu.test_reports import parse_junit_report, test_health_by_module

st.set_page_config(page_title="garantiu", layout="wide")

if "analysis" not in st.session_state:
    st.session_state.analysis = None

screen = st.sidebar.radio(
    "Tela",
    ["Conectar Release", "Visão Geral do Risco", "Roteiro de Teste Manual", "Decisão de Publicação"],
)

if screen == "Conectar Release":
    st.title("Conectar release")
    repo_path = st.text_input("Caminho do repositório", value=".")
    base_ref = st.text_input("Comparar desde", value="HEAD~1")
    head_ref = st.text_input("Branch do release", value="HEAD")
    junit_path = st.text_input("Relatório de testes (JUnit XML)", value="sample_data/sample_junit.xml")
    incidents_path = st.text_input("Arquivo de incidentes (CSV)", value="sample_data/incidents.csv")

    if st.button("Analisar mudanças"):
        changed_files = get_changed_files(repo_path, base_ref, head_ref)
        bug_history = build_bug_history(repo_path)
        test_results = parse_junit_report(junit_path)
        test_health = test_health_by_module(test_results)
        incidents = load_incidents(incidents_path)

        # flakiness real entra na Task 15; até lá, roda sem esse sub-sinal.
        module_scores = score_modules(changed_files, bug_history, test_health, incidents, flakiness={})
        release = score_release(module_scores)

        st.session_state.analysis = {
            "release_name": head_ref,
            "changed_files": changed_files,
            "module_scores": module_scores,
            "release": release,
        }
        st.success(f"{len(changed_files)} arquivo(s) analisado(s) em {len(module_scores)} módulo(s).")

elif screen == "Visão Geral do Risco":
    st.title("Visão geral do risco")
    analysis = st.session_state.analysis
    if not analysis:
        st.info("Analise um release na tela 'Conectar Release' primeiro.")
    else:
        release = analysis["release"]
        st.metric("Score do release", f"{release['score']:.0f}/100")
        st.caption(f"Puxado pelo módulo: {release['top_module']}")

        st.subheader("Composição do score (módulo de maior risco)")
        for factor, value in release["factors"].items():
            st.progress(min(value, 100) / 100, text=f"{factor}: {value:.0f}")

        st.subheader("Módulos mais arriscados")
        st.table([
            {"Módulo": m["module"], "Score": m["score"]}
            for m in analysis["module_scores"]
        ])

```

```

    })

elif screen == "Roteiro de Teste Manual":
    st.title("Roteiro de teste manual")
    analysis = st.session_state.analysis
    if not analysis:
        st.info("Analise um release na tela 'Conectar Release' primeiro.")
    else:
        for m in analysis["module_scores"]:
            card = build_module_card(m["module"], m["score"], m["factors"], analysis["changed_files"])
            with st.container(border=True):
                st.subheader(f"{card['module']} – risco {card['risk']}")
                st.write("***O que mudou**")
                st.write(card["o_que_mudou"])
                st.write("***Por que testar isso**")
                st.write(card["por_que_testar"])
                st.write("***Cenários sugeridos**")
                for cenario in card["cenarios"]:
                    st.write(f"- {cenario}")

elif screen == "Decisão de Publicação":
    st.title("Decisão de publicação")
    analysis = st.session_state.analysis
    if not analysis:
        st.info("Analise um release na tela 'Conectar Release' primeiro.")
    else:
        release = analysis["release"]
        st.metric("Score atual", f"{release['score']:.0f}/100")
        decided_by = st.text_input("Seu nome")

        col1, col2 = st.columns(2)
        if col1.button("Publicar mesmo assim") and decided_by:
            record_decision("garantiu.db", analysis["release_name"], release["score"], decided_by, "publicar")
            st.success("Decisão registrada: publicar.")
        if col2.button("Cancelar publicação") and decided_by:
            record_decision("garantiu.db", analysis["release_name"], release["score"], decided_by, "cancelar")
            st.warning("Decisão registrada: cancelar.")

    st.subheader("Registro (auditoria)")
    history = get_decision_history("garantiu.db", analysis["release_name"])
    st.table(history)

```

- ☐ **Step 5: Rodar o teste de fumaça e confirmar que passa**

Run: `pytest tests/test_app_smoke.py -v` Expected: 1 passed

- ☐ **Step 6: Rodar a suíte inteira**

Run: `pytest -v` Expected: todos os testes das Tasks 1–8 passando (as Tasks 9–15 adicionam mais testes depois).

- ☐ **Step 7: Rodar a aplicação manualmente pra ver funcionando**

Run: `streamlit run app.py` Expected: abre no navegador; na tela "Conectar Release", usar `repo_path="."`, `base_ref` apontando pra um commit anterior do próprio repositório `garantiu` e clicar "Analisar mudanças" mostra dados reais nas telas seguintes.

- ☐ **Step 8: Commit**

```

git add app.py sample_data/incidents.csv sample_data/sample_junit.xml tests/test_app_smoke.py
git commit -m "feat: wire the 4 MVP screens together in a Streamlit app"

```

Task 9: Histórico de execuções de teste + flakiness real

Files: - Create: `garantiu/test_history.py` - Test: `tests/test_test_history.py`

Interfaces: - Consumes: a saída de `parse_junit_report` (Task 3) — mesma lista de dicts com `"classname"`, `"name"`, `"status"`. - Produces: `record_test_run(db_path, test_results) -> None`; `flakiness_by_module(db_path) -> dict[str, float]`.

- ❑ **Step 1: Escrever o teste que falha**

`tests/test_test_history.py`:

```
import pytest

from garantiu.test_history import flakiness_by_module, record_test_run

@pytest.fixture
def db_path(tmp_path):
    return str(tmp_path / "history.db")

def test_flakiness_by_module_needs_at_least_two_runs(db_path):
    run1 = [{"classname": "checkout.test_gateway", "name": "test_a", "status": "passed", "time": 0.01}]
    record_test_run(db_path, run1)
    assert flakiness_by_module(db_path) == {"checkout": 0.0}

def test_flakiness_by_module_detects_flips(db_path):
    record_test_run(db_path, [{"classname": "checkout.test_gateway", "name": "test_a", "status": "passed", "time": 0.01}])
    record_test_run(db_path, [{"classname": "checkout.test_gateway", "name": "test_a", "status": "failed", "time": 0.01}])
    record_test_run(db_path, [{"classname": "checkout.test_gateway", "name": "test_a", "status": "passed", "time": 0.01}])

    result = flakiness_by_module(db_path)
    assert result["checkout"] == 100.0

def test_flakiness_by_module_stable_test_is_zero(db_path):
    for _ in range(3):
        record_test_run(db_path, [{"classname": "auth.test_sessao", "name": "test_b", "status": "passed", "time": 0.01}])
    assert flakiness_by_module(db_path)["auth"] == 0.0
```

- ❑ **Step 2: Rodar o teste e confirmar que falha**

Run: `pytest tests/test_test_history.py -v` Expected: FAIL com `ModuleNotFoundError`

- ❑ **Step 3: Implementar** `garantiu/test_history.py`

```

import sqlite3
from datetime import datetime, timezone

def init_db(db_path: str) -> None:
    conn = sqlite3.connect(db_path)
    try:
        conn.execute(
            """
            CREATE TABLE IF NOT EXISTS test_runs (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                module TEXT NOT NULL,
                classname TEXT NOT NULL,
                name TEXT NOT NULL,
                status TEXT NOT NULL,
                recorded_at TEXT NOT NULL
            )
            """
        )
        conn.commit()
    finally:
        conn.close()

def record_test_run(db_path: str, test_results: list) -> None:
    """Stores one run's results. module = first segment of classname."""
    init_db(db_path)
    conn = sqlite3.connect(db_path)
    try:
        recorded_at = datetime.now(timezone.utc).isoformat()
        for r in test_results:
            module = r["classname"].split(".")[0]
            conn.execute(
                "INSERT INTO test_runs (module, classname, name, status, recorded_at) "
                "VALUES (?, ?, ?, ?, ?)",
                (module, r["classname"], r["name"], r["status"], recorded_at),
            )
        conn.commit()
    finally:
        conn.close()

def flakiness_by_module(db_path: str) -> dict:
    """
    For each test (classname+name), counts flips between consecutive
    recorded statuses (ordered by insertion). A test's flakiness rate =
    flips / (runs - 1), 0 if fewer than 2 runs. Returns {module: average
    flakiness rate * 100}, rounded to 1 decimal.
    """
    init_db(db_path)
    conn = sqlite3.connect(db_path)
    conn.row_factory = sqlite3.Row
    try:
        rows = conn.execute(
            "SELECT module, classname, name, status FROM test_runs "
            "ORDER BY classname, name, id"
        ).fetchall()
    finally:
        conn.close()

    by_test: dict = {}
    for row in rows:
        key = (row["classname"], row["name"])
        by_test.setdefault(key, {"module": row["module"], "statuses": []})
        by_test[key]["statuses"].append(row["status"])

```

```

module_rates: dict = {}
for data in by_test.values():
    statuses = data["statuses"]
    if len(statuses) < 2:
        rate = 0.0
    else:
        flips = sum(1 for a, b in zip(statuses, statuses[1:]) if a != b)
        rate = flips / (len(statuses) - 1)
    module_rates.setdefault(data["module"], []).append(rate)

return {
    module: round(100 * sum(rates) / len(rates), 1)
    for module, rates in module_rates.items()
}

```

- ☐ **Step 4: Rodar o teste e confirmar que passa**

Run: `pytest tests/test_test_history.py -v` Expected: 3 passed

- ☐ **Step 5: Commit**

```

git add garantiu/test_history.py tests/test_test_history.py
git commit -m "feat: record test run history and compute real flakiness per module"

```

Task 10: Priorização da suíte automatizada (tela 4)

Files: - Create: `garantiu/test_prioritization.py` - Test: `tests/test_test_prioritization.py`

Interfaces: - Consumes: a saída de `parse_junit_report` (Task 3), `score_modules` (Task 5), `flakiness_by_module` (Task 9). - Produces: `prioritize_tests(test_results, module_scores, flakiness) -> list[dict]`.

- ☐ **Step 1: Escrever o teste que falha**

`tests/test_test_prioritization.py` :


```

from garantiu.test_prioritization import prioritize_tests

def test_prioritize_tests_orders_by_module_score_then_status():
    test_results = [
        {"name": "test_busca", "classname": "catalogo.test_busca", "status": "passed", "time": 0.01},
        {"name": "test_pagamento_recusado", "classname": "checkout.test_gateway", "status": "failed", "time": 0.02},
        {"name": "test_pagamento_aprovado", "classname": "checkout.test_gateway", "status": "passed", "time": 0.01},
    ]
    module_scores = [
        {"module": "checkout", "score": 85.0, "factors": {}},
        {"module": "catalogo", "score": 10.0, "factors": {}},
    ]
    flakiness = {"checkout": 20.0}

    ordered = prioritize_tests(test_results, module_scores, flakiness)

    assert [t["name"] for t in ordered] == [
        "test_pagamento_recusado",
        "test_pagamento_aprovado",
        "test_busca",
    ]
    assert ordered[0]["flakiness"] == 20.0
    assert ordered[0]["module_score"] == 85.0

def test_prioritize_tests_breaks_ties_by_flakiness_then_name():
    test_results = [
        {"name": "test_b", "classname": "checkout.test_x", "status": "passed", "time": 0.01},
        {"name": "test_a", "classname": "checkout.test_y", "status": "passed", "time": 0.01},
    ]
    module_scores = [{"module": "checkout", "score": 50.0, "factors": {}}]
    flakiness = {"checkout": 0.0}

    ordered = prioritize_tests(test_results, module_scores, flakiness)
    assert [t["name"] for t in ordered] == ["test_a", "test_b"]

```

- ☐ **Step 2: Rodar o teste e confirmar que falha**

Run: `pytest tests/test_test_prioritization.py -v` Expected: FAIL com `ModuleNotFoundError`

- ☐ **Step 3: Implementar** `garantiu/test_prioritization.py`

```
def prioritize_tests(test_results: list, module_scores: list, flakiness: dict) -> list:
    """
    Enriches each test with its module and risk contribution, then sorts
    for the "Suíte Automatizada Priorizada" screen: (1) module score desc,
    (2) failed before passed/skipped, (3) flakiness desc, (4) name asc.
    """
    score_by_module = {m["module"]: m["score"] for m in module_scores}

    enriched = []
    for r in test_results:
        module = r["classname"].split(".")[0]
        enriched.append({
            "name": r["name"],
            "classname": r["classname"],
            "module": module,
            "status": r["status"],
            "time": r["time"],
            "flakiness": flakiness.get(module, 0.0),
            "module_score": score_by_module.get(module, 0.0),
        })

    status_rank = {"failed": 0, "skipped": 1, "passed": 2}

    def sort_key(t):
        return (-t["module_score"], status_rank.get(t["status"], 2), -t["flakiness"], t["name"])

    return sorted(enriched, key=sort_key)
```

- ☐ **Step 4: Rodar o teste e confirmar que passa**

Run: `pytest tests/test_test_prioritization.py -v` Expected: 2 passed

- ☐ **Step 5: Commit**

```
git add garantiu/test_prioritization.py tests/test_test_prioritization.py
git commit -m "feat: prioritize automated test suite by module risk"
```

Task 11: Detalhe de bugs e incidentes por módulo

Files: - Modify: `garantiu/bug_history.py` (adiciona uma função) - Modify: `garantiu/incidents.py` (adiciona uma função) - Modify: `tests/test_bug_history.py` (adiciona testes) - Modify: `tests/test_incidents.py` (adiciona testes) - Create: `sample_data/incident_details.csv`

Interfaces: - Consumes: `get_bug_fix_commits` (Task 2, já existe no mesmo arquivo). - Produces: `bug_history_detail_by_module(repo_path, module) -> list[dict]` (chaves "hash", "message", "date"); `load_incident_details(csv_path) -> dict[str, list[dict]]` (cada item "description", "date").

- ☐ **Step 1: Escrever os testes que falham**

Adicionar ao final de `tests/test_bug_history.py` (e trocar a linha de import no topo do arquivo para `from garantiu.bug_history import bug_history_detail_by_module, build_bug_history, get_bug_fix_commits`):

```
def test_bug_history_detail_by_module_lists_commits_most_recent_first(repo_with_bug_fixes):
    details = bug_history_detail_by_module(repo_with_bug_fixes, "checkout")
    assert len(details) == 2
    assert details[0]["message"].startswith("fixes bug")
    assert details[1]["message"].startswith("fix:")
    assert all("date" in d for d in details)
```

Adicionar ao final de `tests/test_incidents.py` (e trocar a linha de import no topo para `from garantiu.incidents import load_incident_details, load_incidents`):

```
def test_load_incident_details_groups_and_sorts_by_date(tmp_path):
    path = tmp_path / "incident_details.csv"
    path.write_text(
        "module,description,date\n"
        "checkout,gateway fora do ar 22 min,2026-02-10\n"
        "checkout,cobranca duplicada,2026-03-05\n"
        "auth,sessao nao expirava,2026-01-20\n"
    )
    details = load_incident_details(str(path))
    assert [d["date"] for d in details["checkout"]] == ["2026-03-05", "2026-02-10"]
    assert details["auth"][0]["description"] == "sessao nao expirava"
```

- ❑ **Step 2: Rodar os testes e confirmar que falham**

Run: `pytest tests/test_bug_history.py tests/test_incidents.py -v` Expected: FAIL com `ImportError` (as novas funções ainda não existem)

- ❑ **Step 3: Adicionar a `garantiu/bug_history.py`**

```
def bug_history_detail_by_module(repo_path: str, module: str) -> list:
    """
    Returns bug-fix commits that touched any file under `module`, as
    {"hash", "message", "date"} (ISO date), most recent first. A commit
    that touched multiple files in the module appears once.
    """
    repo = git.Repo(repo_path)
    seen = set()
    results = []
    for commit_data in get_bug_fix_commits(repo_path):
        touches_module = any(f.split("/")[0] == module for f in commit_data["files"])
        if touches_module and commit_data["hash"] not in seen:
            seen.add(commit_data["hash"])
            commit = repo.commit(commit_data["hash"])
            results.append({
                "hash": commit_data["hash"],
                "message": commit_data["message"],
                "date": commit.committed_datetime.date().isoformat(),
            })
    results.sort(key=lambda c: c["date"], reverse=True)
    return results
```

(`git` já está importado no topo do arquivo desde a Task 2.)

- ❑ **Step 4: Adicionar a `garantiu/incidents.py`**

```

from collections import defaultdict

def load_incident_details(csv_path: str) -> dict:
    """
    Reads a CSV with columns 'module', 'description', 'date' (one row per
    incident) and returns {module: [{"description": "date"}, ...]}, each
    module's list sorted most recent first.
    """
    by_module = defaultdict(list)
    with open(csv_path, newline="", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        for row in reader:
            by_module[row["module"]].append({
                "description": row["description"],
                "date": row["date"],
            })
    for entries in by_module.values():
        entries.sort(key=lambda i: i["date"], reverse=True)
    return dict(by_module)

```

(csv já está importado no topo do arquivo desde a Task 4; adicionar `from collections import defaultdict` junto dos imports existentes.)

- ☐ **Step 5: Criar** `sample_data/incident_details.csv`

```

module,description,date
checkout,gateway fora do ar 22 min,2026-02-14
checkout,cobranca duplicada em parcelamento,2026-03-05
auth,sessao nao expirava corretamente,2026-01-20

```

- ☐ **Step 6: Rodar os testes e confirmar que passam**

Run: `pytest tests/test_bug_history.py tests/test_incidents.py -v` Expected: todos passando

- ☐ **Step 7: Commit**

```

git add garantiu/bug_history.py garantiu/incidents.py tests/test_bug_history.py tests/test_incidents.py sample_data/incident_details.csv
git commit -m "feat: add per-module bug and incident detail lookups"

```

Task 12: Agregador de detalhe do módulo (tela 5)

Files: - Create: `garantiu/module_detail.py` - Test: `tests/test_module_detail.py`

Interfaces: - Consumes: `get_changed_files` (Task 1), `bug_history_detail_by_module` (Task 11), `load_incident_details` (Task 11), `test_health_by_module` (Task 3), `flakiness_by_module` (Task 9). - Produces: `build_module_detail(module, changed_files, bug_details, incident_details, test_health, flakiness) -> dict`.

- ☐ **Step 1: Escrever o teste que falha**

`tests/test_module_detail.py` :

```

from garantiu.module_detail import build_module_detail

def test_build_module_detail_combines_everything():
    changed_files = [
        {"path": "checkout/gateway.py", "module": "checkout", "lines_added": 10, "lines_removed": 2},
        {"path": "auth/session.py", "module": "auth", "lines_added": 1, "lines_removed": 0},
    ]
    bug_details = [{"hash": "abc123", "message": "fix: x", "date": "2026-03-05"}]
    incident_details = [{"description": "gateway fora do ar", "date": "2026-02-10"}]
    test_health = {"checkout": 60.0}
    flakiness = {"checkout": 20.0}

    detail = build_module_detail("checkout", changed_files, bug_details, incident_details, test_health, flakiness)

    assert len(detail["files"]) == 1
    assert detail["files"][0]["path"] == "checkout/gateway.py"
    assert detail["bugs"] == bug_details
    assert detail["incidents"] == incident_details
    assert detail["test_health"] == 60.0
    assert detail["flakiness"] == 20.0

def test_build_module_detail_defaults_missing_test_data_to_healthy():
    detail = build_module_detail("novo", [], [], [], {}, {})
    assert detail["test_health"] == 100.0
    assert detail["flakiness"] == 0.0

```

- ☐ **Step 2: Rodar o teste e confirmar que falha**

Run: `pytest tests/test_module_detail.py -v` Expected: FAIL com `ModuleNotFoundError`

- ☐ **Step 3: Implementar** `garantiu/module_detail.py`

```

def build_module_detail(module: str, changed_files: list, bug_details: list, incident_details: list, test_health: dict)
    """Combines everything the "Detalhe do Módulo" screen needs into one dict."""
    files_in_module = [f for f in changed_files if f["module"] == module]
    return {
        "module": module,
        "files": files_in_module,
        "bugs": bug_details,
        "incidents": incident_details,
        "test_health": test_health.get(module, 100.0),
        "flakiness": flakiness.get(module, 0.0),
    }

```

- ☐ **Step 4: Rodar o teste e confirmar que passa**

Run: `pytest tests/test_module_detail.py -v` Expected: 2 passed

- ☐ **Step 5: Commit**

```

git add garantiu/module_detail.py tests/test_module_detail.py
git commit -m "feat: aggregate module detail data for the module detail screen"

```

Task 13: Histórico de releases — score previsto x resultado real (tela 7)

Files: - Create: `garantiu/release_history.py` - Test: `tests/test_release_history.py`

Interfaces: - Produces: `record_release_score(db_path, release, score) -> int`; `record_release_outcome(db_path, release, outcome) -> int`; `get_release_history(db_path) -> list[dict]`.

- ☐ **Step 1: Escrever o teste que falha**

`tests/test_release_history.py` :

```
import pytest

from garantiu.release_history import get_release_history, record_release_outcome, record_release_score

@pytest.fixture
def db_path(tmp_path):
    return str(tmp_path / "releases.db")

def test_record_release_outcome_rejects_invalid_value(db_path):
    with pytest.raises(ValueError):
        record_release_outcome(db_path, "release/2026.08", "talvez")

def test_get_release_history_joins_score_and_outcome(db_path):
    record_release_score(db_path, "release/2026.08", 58.0)
    record_release_outcome(db_path, "release/2026.08", "ok")
    record_release_score(db_path, "release/2026.09", 85.0)

    history = get_release_history(db_path)
    by_release = {h["release"]: h for h in history}

    assert by_release["release/2026.08"]["outcome"] == "ok"
    assert by_release["release/2026.09"]["outcome"] is None
    assert by_release["release/2026.09"]["score"] == 85.0
```

- ☐ **Step 2: Rodar o teste e confirmar que falha**

Run: `pytest tests/test_release_history.py -v` Expected: FAIL com `ModuleNotFoundError`

- ☐ **Step 3: Implementar** `garantiu/release_history.py`

```

import sqlite3
from datetime import datetime, timezone

def init_db(db_path: str) -> None:
    conn = sqlite3.connect(db_path)
    try:
        conn.execute(
            """
            CREATE TABLE IF NOT EXISTS release_scores (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                release TEXT NOT NULL,
                score REAL NOT NULL,
                computed_at TEXT NOT NULL
            )
            """
        )
        conn.execute(
            """
            CREATE TABLE IF NOT EXISTS release_outcomes (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                release TEXT NOT NULL,
                outcome TEXT NOT NULL,
                recorded_at TEXT NOT NULL
            )
            """
        )
        conn.commit()
    finally:
        conn.close()

def record_release_score(db_path: str, release: str, score: float) -> int:
    init_db(db_path)
    conn = sqlite3.connect(db_path)
    try:
        cursor = conn.execute(
            "INSERT INTO release_scores (release, score, computed_at) VALUES (?, ?, ?)",
            (release, score, datetime.now(timezone.utc).isoformat()),
        )
        conn.commit()
        return cursor.lastrowid
    finally:
        conn.close()

def record_release_outcome(db_path: str, release: str, outcome: str) -> int:
    """outcome must be 'ok' or 'falhou'."""
    if outcome not in ("ok", "falhou"):
        raise ValueError("outcome must be 'ok' or 'falhou'")
    init_db(db_path)
    conn = sqlite3.connect(db_path)
    try:
        cursor = conn.execute(
            "INSERT INTO release_outcomes (release, outcome, recorded_at) VALUES (?, ?, ?)",
            (release, outcome, datetime.now(timezone.utc).isoformat()),
        )
        conn.commit()
        return cursor.lastrowid
    finally:
        conn.close()

def get_release_history(db_path: str) -> list:
    """

```

```

Returns one entry per scored release, most recent first:
{"release", "score", "computed_at", "outcome"}. outcome is None if no
outcome has been recorded yet.
"""

init_db(db_path)
conn = sqlite3.connect(db_path)
conn.row_factory = sqlite3.Row
try:
    scores = conn.execute(
        "SELECT release, score, computed_at FROM release_scores ORDER BY computed_at DESC"
    ).fetchall()
    outcomes = conn.execute(
        "SELECT release, outcome FROM release_outcomes ORDER BY recorded_at DESC"
    ).fetchall()
finally:
    conn.close()

latest_outcome = {}
for row in outcomes:
    latest_outcome.setdefault(row["release"], row["outcome"])

return [
    {
        "release": row["release"],
        "score": row["score"],
        "computed_at": row["computed_at"],
        "outcome": latest_outcome.get(row["release"]),
    }
    for row in scores
]

```

- ☐ **Step 4: Rodar o teste e confirmar que passa**

Run: `pytest tests/test_release_history.py -v` Expected: 2 passed

- ☐ **Step 5: Commit**

```

git add garantiu/release_history.py tests/test_release_history.py
git commit -m "feat: record and query release score vs real outcome history"

```

Task 14: Ligar as 3 telas restantes + flakiness/histórico de release em `app.py`

Files: - Modify: `app.py` (versão final, substitui o conteúdo da Task 8) - Modify: `tests/test_app_smoke.py` (versão final, substitui o conteúdo da Task 8)

Interfaces: - Consumes: todas as funções das Tasks 1–13.

- ☐ **Step 1: Escrever o teste que falha**

Substituir todo o conteúdo de `tests/test_app_smoke.py` :


```

from streamlit.testing.v1 import AppTest

def test_app_loads_without_exceptions():
    at = AppTest.from_file("app.py")
    at.run()
    assert not at.exception

def test_app_loads_all_screens_without_exceptions():
    screens = [
        "Conectar Release",
        "Visão Geral do Risco",
        "Roteiro de Teste Manual",
        "Suíte Automatizada Priorizada",
        "Detalhe do Módulo",
        "Decisão de Publicação",
        "Histórico & Tendências",
    ]
    for screen in screens:
        at = AppTest.from_file("app.py")
        at.run()
        at.sidebar.radio[0].set_value(screen).run()
        assert not at.exception

```

- ☐ **Step 2: Rodar o teste e confirmar que falha**

Run: `pytest tests/test_app_smoke.py -v` Expected: FAIL — `set_value("Suíte Automatizada Priorizada")` não encontra essa opção no radio atual (só existem as 4 telas da Task 8).

- ☐ **Step 3: Substituir todo o conteúdo de `app.py`**

```

import streamlit as st

from garantiu.bug_history import bug_history_detail_by_module, build_bug_history
from garantiu.decision_log import get_decision_history, record_decision
from garantiu.git_reader import get_changed_files
from garantiu.incidents import load_incident_details, load_incidents
from garantiu.manual_test_guide import build_module_card
from garantiu.module_detail import build_module_detail
from garantiu.release_history import get_release_history, record_release_outcome, record_release_score
from garantiu.scoring import score_modules, score_release
from garantiu.test_history import flakiness_by_module, record_test_run
from garantiu.test_prioritization import prioritize_tests
from garantiu.test_reports import parse_junit_report, test_health_by_module

DECISIONS_DB = "garantiu.db"
TEST_HISTORY_DB = "garantiu_test_history.db"
RELEASE_HISTORY_DB = "garantiu_release_history.db"

st.set_page_config(page_title="garantiu", layout="wide")

if "analysis" not in st.session_state:
    st.session_state.analysis = None

screen = st.sidebar.radio(
    "Tela",
    [
        "Conectar Release",
        "Visão Geral do Risco",
        "Roteiro de Teste Manual",
        "Suíte Automatizada Priorizada",
        "Detalhe do Módulo",
        "Decisão de Publicação",
        "Histórico & Tendências",
    ],
)

if screen == "Conectar Release":
    st.title("Conectar release")
    repo_path = st.text_input("Caminho do repositório", value=".")
    base_ref = st.text_input("Comparar desde", value="HEAD~1")
    head_ref = st.text_input("Branch do release", value="HEAD")
    junit_path = st.text_input("Relatório de testes (JUnit XML)", value="sample_data/sample_junit.xml")
    incidents_path = st.text_input("Arquivo de incidentes (CSV)", value="sample_data/incidents.csv")
    incident_details_path = st.text_input(
        "Arquivo de detalhe de incidentes (CSV)", value="sample_data/incident_details.csv"
    )

    if st.button("Analisar mudanças"):
        changed_files = get_changed_files(repo_path, base_ref, head_ref)
        bug_history = build_bug_history(repo_path)
        test_results = parse_junit_report(junit_path)
        test_health = test_health_by_module(test_results)
        incidents = load_incidents(incidents_path)

        record_test_run(TEST_HISTORY_DB, test_results)
        flakiness = flakiness_by_module(TEST_HISTORY_DB)

        module_scores = score_modules(changed_files, bug_history, test_health, incidents, flakiness)
        release = score_release(module_scores)
        record_release_score(RELEASE_HISTORY_DB, head_ref, release["score"])

    st.session_state.analysis = {
        "release_name": head_ref,
        "repo_path": repo_path,
        "incident_details_path": incident_details_path,
    }

```

```

        "changed_files": changed_files,
        "module_scores": module_scores,
        "release": release,
        "test_results": test_results,
        "test_health": test_health,
        "flakiness": flakiness,
    }
    st.success(f"{len(changed_files)} arquivo(s) analisado(s) em {len(module_scores)} módulo(s).")

elif screen == "Visão Geral do Risco":
    st.title("Visão geral do risco")
    analysis = st.session_state.analysis
    if not analysis:
        st.info("Analise um release na tela 'Conectar Release' primeiro.")
    else:
        release = analysis["release"]
        st.metric("Score do release", f"{release['score']:.0f}/100")
        st.caption(f"Puxado pelo módulo: {release['top_module']}")

        st.subheader("Composição do score (módulo de maior risco)")
        for factor, value in release["factors"].items():
            st.progress(min(value, 100) / 100, text=f"{factor}: {value:.0f}")

        st.subheader("Módulos mais arriscados")
        st.table([
            {"Módulo": m["module"], "Score": m["score"]}
            for m in analysis["module_scores"]
        ])

elif screen == "Roteiro de Teste Manual":
    st.title("Roteiro de teste manual")
    analysis = st.session_state.analysis
    if not analysis:
        st.info("Analise um release na tela 'Conectar Release' primeiro.")
    else:
        for m in analysis["module_scores"]:
            card = build_module_card(m["module"], m["score"], m["factors"], analysis["changed_files"])
            with st.container(border=True):
                st.subheader(f"{card['module']} – risco {card['risk']}")
                st.write("***O que mudou**")
                st.write(card["o_que_mudou"])
                st.write("***Por que testar isso**")
                st.write(card["por_que_testar"])
                st.write("***Cenários sugeridos**")
                for cenario in card["cenarios"]:
                    st.write(f"- {cenario}")

elif screen == "Suíte Automatizada Priorizada":
    st.title("Suíte automatizada priorizada")
    analysis = st.session_state.analysis
    if not analysis:
        st.info("Analise um release na tela 'Conectar Release' primeiro.")
    else:
        ordered = prioritize_tests(analysis["test_results"], analysis["module_scores"], analysis["flakiness"])
        st.table([
            {
                "Teste": t["name"],
                "Módulo": t["module"],
                "Status": t["status"],
                "Flakiness (%)": t["flakiness"],
                "Score do módulo": t["module_score"],
            }
            for t in ordered
        ])

```

```

elif screen == "Detalhe do Módulo":
    st.title("Detalhe do módulo")
    analysis = st.session_state.analysis
    if not analysis:
        st.info("Analise um release na tela 'Conectar Release' primeiro.")
    else:
        modules = [m["module"] for m in analysis["module_scores"]]
        selected = st.selectbox("Módulo", modules)
        bug_details = bug_history_detail_by_module(analysis["repo_path"], selected)
        incident_details = load_incident_details(analysis["incident_details_path"]).get(selected, [])
        detail = build_module_detail(
            selected,
            analysis["changed_files"],
            bug_details,
            incident_details,
            analysis["test_health"],
            analysis["flakiness"],
        )

        st.subheader("O que mudou")
        st.table([{"Arquivo": f["path"], "+": f["lines_added"], "-": f["lines_removed"]} for f in detail["files"]])

        st.subheader("Histórico de bugs")
        if detail["bugs"]:
            st.table(detail["bugs"])
        else:
            st.caption("Nenhum bug histórico registrado pra esse módulo.")

        st.subheader("Incidentes em produção")
        if detail["incidents"]:
            st.table(detail["incidents"])
        else:
            st.caption("Nenhum incidente registrado pra esse módulo.")

        st.subheader("Saúde dos testes")
        st.write(f"Taxa de aprovação na última rodada: {detail['test_health']:.0f}%")
        st.write(f"Flakiness histórica: {detail['flakiness']:.0f}%")

elif screen == "Decisão de Publicação":
    st.title("Decisão de publicação")
    analysis = st.session_state.analysis
    if not analysis:
        st.info("Analise um release na tela 'Conectar Release' primeiro.")
    else:
        release = analysis["release"]
        st.metric("Score atual", f"{release['score']:.0f}/100")
        decided_by = st.text_input("Seu nome")

        col1, col2 = st.columns(2)
        if col1.button("Publicar mesmo assim") and decided_by:
            record_decision(DECISIONS_DB, analysis["release_name"], release["score"], decided_by, "publicar")
            st.success("Decisão registrada: publicar.")
        if col2.button("Cancelar publicação") and decided_by:
            record_decision(DECISIONS_DB, analysis["release_name"], release["score"], decided_by, "cancelar")
            st.warning("Decisão registrada: cancelar.")

        st.subheader("Registro (auditoria)")
        history = get_decision_history(DECISIONS_DB, analysis["release_name"])
        st.table(history)

elif screen == "Histórico & Tendências":
    st.title("Histórico & tendências")
    history = get_release_history(RELEASE_HISTORY_DB)
    if not history:
        st.info("Ainda não há releases analisados. Use a tela 'Conectar Release' primeiro.")

```

```

else:
    st.table(history)

    st.subheader("Marcar resultado real de um release")
    release_options = [h["release"] for h in history]
    release_to_mark = st.selectbox("Release", release_options)
    outcome = st.radio("Resultado", ["ok", "falhou"], horizontal=True)
    if st.button("Registrar resultado"):
        record_release_outcome(RELEASE_HISTORY_DB, release_to_mark, outcome)
        st.success(f"Resultado de {release_to_mark} registrado como '{outcome}'.")
        st.rerun()

```

- ☐ **Step 4: Rodar o teste e confirmar que passa**

Run: `pytest tests/test_app_smoke.py -v` Expected: 2 passed

- ☐ **Step 5: Rodar a suíte inteira**

Run: `pytest -v` Expected: todos os testes das Tasks 1–14 passando.

- ☐ **Step 6: Rodar a aplicação manualmente pra ver as 7 telas funcionando**

Run: `streamlit run app.py` Expected: na tela "Conectar Release", `repo_path="."`, um `base_ref` que exista no histórico do próprio repositório `garantiu` e "Analisar mudanças" preenche as 7 telas com dado real. Repita a análise (mesmo release ou outro) pra ver a flakiness e o histórico de releases deixarem de ser o/vazio.

- ☐ **Step 7: Commit**

```

git add app.py tests/test_app_smoke.py
git commit -m "feat: wire remaining 3 screens (suite, module detail, history) into the app"

```

Self-Review

Cobertura do escopo: as 7 telas do wireframe têm cada uma uma seção correspondente em `app.py` (Task 8 monta 4, Task 14 monta as 3 restantes e substitui o arquivo inteiro). O algoritmo do score (Task 5, revisado) implementa a fórmula acordada — 4 fatores, pesos iguais, rollout por `max()` — agora com `saude_testes` combinando taxa de falha atual e flakiness histórica real (Task 9), em vez do placeholder de rodada única do primeiro rascunho do plano.

Placeholders: nenhum `TODO` / `TBD` — toda função tem implementação completa e testada, incluindo as adições das Tasks 9–14.

Consistência de tipos entre tasks: `score_modules` (Task 5) agora recebe `flakiness` como 5º parâmetro — a chamada em `app.py` (Task 8) foi atualizada para passar `flakiness={}` até a Task 9 existir, e a versão final de `app.py` (Task 14) passa o `flakiness_by_module()` real. `prioritize_tests` (Task 10), `build_module_detail` (Task 12) e o `app.py` final (Task 14) usam as mesmas chaves (`"module"`, `"score"`, `"factors"`, `"classname"`, `"status"`) definidas nas Tasks 1, 3 e 5.

Plan complete and saved to `docs/superpowers/plans/2026-09-09-garantiu-mvp-implementation.md` . **Duas opções de execução:**

1. **Subagent-Driven (recomendado)** — eu despacho um subagente novo por task, com revisão entre elas, iteração rápida.
2. **Execução Inline** — eu executo as tasks nesta sessão, em lote, com checkpoints pra revisão.

Qual abordagem?